

INT 221 - MVC PROGRAMMING

Unit 3

Controller

- In Laravel, a controller is a crucial component used to handle HTTP requests and define the application's response logic. Controllers serve as an intermediary between routes and the actual business logic of your application.
- Instead of defining all of your request handling logic as closures in your route files, you may wish to organize this behavior using "controller" classes.
- Controllers can group related request handling logic into a single class.

Controller

- For example, a UserController class might handle all incoming requests related to users, including showing, creating, updating, and deleting users.
- By default, controllers are stored in the app/Http/Controllers directory.
- A **Controller** handles request logic and connects:
- Route → Controller → View/Response
- Keeps code clean (follows MVC pattern)
- Avoids writing logic in routes

php artisan make:controller StudentController

File created at: app/Http/Controllers/StudentController.php

Basic Controller Structure

```
namespace App\Http\Controllers;
```

```
use Illuminate\Http\Request;
```

```
class StudentController extends Controller
{
    public function index()
    {
        return "All Students";
    }
}
```

index() → method (action)

Controller with Route

- In `routes/web.php`

```
use App\Http\Controllers\StudentController;  
Route::get('/students', [StudentController::class, 'index']);
```

make sure project is running: `php artisan serve`

- Now open:

<http://localhost:8000/students>

- Output:

All Students

Passing Data from Controller to View

- **Controller:**

```
public function index()
{
    $students = [
        ['id' => 1, 'name' => 'Ravi'],
        ['id' => 2, 'name' => 'Aman']
    ];
    return view('students', compact('students'));
}
```

- **View (`resources/views/students.blade.php`)**

```
@foreach($students as $student)
    <p>{{ $student['name'] }}</p>
@endforeach
```

- Data passed using `compact()`

Dynamic Route + Controller (Single Student)

- **Route:** `Route::get('/student/{id}', [StudentController::class, 'show']);`
- **Controller:**

```
public function show($id)
{
    $students = [
        ['id' => 1, 'name' => 'Ravi'],
        ['id' => 2, 'name' => 'Aman']
    ];
    foreach ($students as $student) {
        if ($student['id'] == $id) {
            return view('student-detail', compact('student'));
        }
    }
    return "Student not found";
}
```

Fetch specific student using ID: like 127.0.0.1:8000/student/1

Multiple Methods in Controller

```
class StudentController extends Controller
```

```
{  
    public function index()  
    {  
        return "All Students";  
    }  
  
    public function create()  
    {  
        return "Create Student Form";  
    }  
  
    public function store(Request $request)  
    {  
        return "Student Saved";  
    }  
}
```

Common CRUD methods

Resource Controller

Creation: `php artisan make:controller StudentController --resource`

- Auto-generates:

```
index()    // show all
create()   // form
store()    // save
show()     // single
edit()     // edit form
update()   // update
destroy()  // delete
```

- **Route:**

```
Route::resource('students', StudentController::class);
```

Automatically creates all routes

Middleware

Middleware is a mechanism that acts as a **filter for HTTP requests** entering your application. It allows you to inspect, modify, allow, or deny requests **before they reach the controller**, and also modify responses **before they are sent back to the user**. Middleware is a layer between **request and response** that checks conditions like authentication, validation, or permissions before allowing access to routes.

Working Flow

Request → Middleware → Controller → Response → Middleware → Browser

age.check → apply on complete project (global middleware)

country.check → apply on specific route + route group

STEP 1: Create Middleware

Run commands:

```
php artisan make:middleware Agecheck
```

```
php artisan make:middleware Countrycheck
```

Middleware

STEP 2: Write Agecheck (GLOBAL)

`app/Http/Middleware/Agecheck.php`

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class Agecheck{
    public function handle(Request $request, Closure $next){
        $age = $request->age;
        if ($age < 18) {
            return "Not allowed (Age < 18)";
        }
        return $next($request);
    }
}
```

Middleware

STEP 3: Write Countrycheck (ROUTE)

`app/Http/Middleware/Countrycheck.php`

```
<?php
namespace App\Http\Middleware;
use Closure;
use Illuminate\Http\Request;
class Countrycheck{
    public function handle(Request $request, Closure $next){
        $country = $request->country;
        if ($country != "India") {
            return "Access denied (Only India allowed)";
        }
        return $next($request);
    }
}
```

Middleware

STEP 4: Register Middleware

`bootstrap/app.php`

```
use Illuminate\Foundation\Configuration\Middleware;

->withMiddleware(function (Middleware $middleware) {
    //Global middleware (runs on all routes)
    $middleware->append(\App\Http\Middleware\Agecheck::class);

    // Alias for route-specific middleware
    $middleware->alias([
        'country.check' => \App\Http\Middleware\Countrycheck::class,
    ]);
});
```

Middleware

STEP 5: Define Routes

Routes/web.php

1. Normal Route (Only Agecheck applies automatically)

```
Route::get('/home', function () {  
    return "Home Page";  
});
```

URL: `http://127.0.0.1:8000/home?age=20`

2. Single Route with Countrycheck

```
Route::get('/profile', function () {  
    return "User Profile";  
})->middleware('country.check');
```

URL: `http://127.0.0.1:8000/profile?age=20&country=India`

`/home?age=15`

Output:

Not allowed (Age < 18)

`/profile?age=20&country=USA`

Output:

Access denied (Only India allowed)

`/profile?age=20&country=India`

Output:

User Profile

Middleware

Route Group with Countrycheck

```
Route::middleware(['country.check']) ->group(function () {  
    Route::get('/dashboard', function () {  
        return "Dashboard";  
    });  
  
    Route::get('/settings', function () {  
        return "Settings";  
    });  
});
```

URLs:

<http://127.0.0.1:8000/dashboard?age=20&country=India>

<http://127.0.0.1:8000/settings?age=20&country=India>

Controller Middleware

- Controller middleware is used to **filter requests before they reach controller methods.**
 - Applied inside controller
 - Controls access (auth, role, etc.)
 - Runs **before method execution**
- Middleware registered in **bootstrap/app.php**
- Used in controller via **`$this->middleware()`**
- **Create Middleware:** php artisan make:middleware CheckRole
- middleware created at: app/Http/Middleware/CheckRole.php

Write Middleware Logic

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
class CheckRole
{
    public function handle(Request $request, Closure $next)
    {
        // Example logic
        if ($request->user() && $request->user()->role === 'admin') {
            return $next($request);
        }
        return redirect('/home')->with('error', 'Unauthorized Access');
    }
}
```

- Allows only admin
- Others redirected

Register Middleware Logic

Open: `bootstrap/app.php`

```
->withMiddleware(function ($middleware) {  
    $middleware->alias([  
        'role' => \App\Http\Middleware\CheckRole::class,  
    ]);  
})
```

✓ Alias name = `role`

Create Controller

```
php artisan make:controller AdminController
```

Apply Middleware in Controller

```
namespace App\Http\Controllers;
class AdminController extends Controller
{
    public function __construct()
    {
        $this->middleware('role');
    }

    public function dashboard()
    {
        return "Admin Dashboard";
    }
}
```

- Middleware applied to all methods

Define Route

```
use App\Http\Controllers\AdminController;
Route::get('/admin', [AdminController::class, 'dashboard']);
```

Demonstration

Case 1: Admin user

```
$request->user()->role = 'admin';
```

- Output:

Admin Dashboard

Case 2: Normal user

```
$request->user()->role = 'user';
```

- Redirect:

/home

- Message:

Unauthorized Access

Controller Middleware Variations

1. Apply to Specific Methods

```
public function __construct(){  
    $this->middleware('role')->only('dashboard');  
}
```

2. Exclude Methods

```
public function __construct(){  
    $this->middleware('role')->except(['index']);  
}
```

3. Multiple Middleware

```
$this->middleware(['auth', 'role']);
```

4. Middleware with Parameter

```
$this->middleware('role:admin');
```

Contd...

5. Inline Middleware

```
public function __construct(){  
    $this->middleware(function ($request, $next) {  
        if (auth()->check()) {  
            return $next($request);  
        }  
  
        return redirect('/login');  
    });  
}
```

FLOW

Request → Route → Controller → Middleware → Method → Response

BLADE TEMPLATES

- Blade is the simple, yet powerful templating engine that is included with Laravel.
- Blade template files use the .blade.php file extension and are typically stored in the resources/views directory.
- You may display data that is passed to your Blade views by wrapping the variable in curly braces. For example, given the following route:

```
Route::get('/', function () {  
    return view('welcome', ['name' => 'Samantha']);  
});
```

- You may display the contents of the name variable like so:

```
Hello, {{ $name }}.
```

Blade Directives

- Blade directives are short PHP snippets that are used to extend the functionality of Blade templates. They are prefixed with an '@' symbol and can be used to perform a variety of tasks, such as:
- **Conditional logic:** The `@if` and `@else` directives can be used to conditionally render output.
- **Loops:** The `@foreach` and `@forelse` directives can be used to loop through collections of data.
- **Including other views:** The `@include` directive can be used to include the contents of another view file.
- **Rendering expressions:** The `@php` and `@endphp` directives can be used to render PHP expressions.

Loops

- Blade provides simple directives for working with PHP's loop structures.

```
@foreach ($users as $user)  
    <p>This is user {{ $user }}</p>  
@endforeach
```

```
@forelse ($users as $user)  
    <li>{{ $user }}</li>  
@empty  
    <p>No users</p>  
@endforelse
```

If Statements

- You may construct if statements using the @if, @elseif, @else, and @endif directives.

```
@if (count($records) === 1)
```

```
    I have one record!
```

```
@elseif (count($records) > 1)
```

```
    I have multiple records!
```

```
@else
```

```
    I don't have any records!
```

```
@endif
```

If Statements (contd.)

- In addition to the conditional directives already discussed, the @isset and @empty directives may be used as convenient shortcuts for their respective PHP functions:

@isset(\$records)

// \$records is defined and is not null...

@endisset

@empty(\$records)

// \$records is "empty"...

@endempty

Switch Statements

- Switch statements can be constructed using the @switch, @case, @break, @default and @endswitch directives:

@switch(\$i)

 @case(1)

 First case...

 @break

 @case(2)

 Second case...

 @break

 @default

 Default case...

@endswitch

Now Some Practical of blade directives

- Step 1 – Make a Controller using Artisan
- **php artisan make:controller UserController**
- Step 2 – Make a route in web.php and don't forget to include the UserController in web.php
- `Route::get('/profile', [UserController::class, 'showProfile']);`
- Step 3, Now in UserController class let's define the function showProfile()

```
public function showProfile() {  
    $username = 'Mr Stark';  
    $isAdmin = true;  
    $items = ['Item 1', 'Item 2', 'Item 3'];  
    return view('profile', compact('username', 'isAdmin',  
'items'));  
}
```

Step 4- define a view in resources/view folder 'profile.blade.php'

```
<!DOCTYPE html>
<html>
  <head>
    <title>User Profile</title>
  </head>
  <body>
    <h1>Welcome to your profile, {{ $username }}</h1>
    @if($isAdmin)
      <p>You are an administrator.</p>
    @else
      <p>You are not an administrator.</p>
    @endif
    <ul>
      @foreach($items as $item)
        <li>{{ $item }}</li>
      @endforeach
    </ul>
```

<hr>

<h2>Additional Blade Directives:</h2>

{{-- @unless directive --}}

@unless(\$isAdmin)

<p>You don't have admin privileges.</p>

@endunless

{{-- @empty directive --}}

@empty(\$additionalData)

<p>Additional data is empty.</p>

@endempty

{{-- @for directive --}}

<h3>Looping with for</h3>

@for(\$i = 1; \$i <= 3; \$i++)

Iteration {{ \$i }}

@endfor


```
    @for($i = 1; $i <= 3; $i++)  
        <li>Iteration {{ $i }}</li>  
    @endfor  
</ul>
```

```
{{-- @include directive --}}  
{{--<h3>Including Sub-Views:</h3>  
@include('partials.footer')--}}
```

```
{{-- Blade Comments --}}  
{{-- This is a Blade comment --}}
```

```
</body>  
</html>
```


What file extension is typically used for Blade template files in Laravel?

- a) .php
 - **b) .blade.php**
 - c) .tpl
 - d) .view
-
- **How can you display a variable named \$name in a Blade template?**
 - a) Hello, \$name.
 - b) Hello, {!! \$name !!}.
 - **c) Hello, {{ \$name }}.**
 - d) Hello, {{\$naame}}.

Which symbol is used to prefix Blade directives in Laravel?

- a) #
- b) \$
- c) !
- d) @

How do you create a loop in a Blade template to iterate through an array called \$users?

- a) @for (\$user in \$users)
- b) @foreach (\$user as \$users)
- c) @loop (\$users as \$user)
- d) @foreach (\$users as \$user)

Which Blade directive is used to check if a variable is set and not null?



- a) **@isset**
- b) @empty
- c) @ifset
- d) @nullcheck

Which Blade directive is used for executing a loop with a defined number of iterations?

- a) @while
- b) @loop
- c) **@for**
- d) @foreach

Template Inheritance

Template Inheritance in Laravel is a **Blade feature** that allows you to create a **master layout** and reuse it across multiple pages.

- It helps avoid code repetition (DRY principle)
- It means:
 - Create one **main layout (parent)**
 - Other pages (**child views**) extend it

Master Layout → Child Views → Final Output

STEP 1: Create Master Layout

`resources/views/layouts/app.blade.php`

```
<!DOCTYPE html>
<html>
  <head>
    <title>@yield('title')</title>
  </head>
  <body>
    <header>
      <h1>My Website</h1>
    </header>
    <nav>
      <a href="/">Home</a>
      <a href="/about">About</a>
    </nav>
    <hr>
    <div>
      @yield('content')
    </div>
    <footer>
      <p>© 2026 My Website</p>
    </footer>
  </body>
</html>
```

`@yield()` → placeholder for child content

STEP 2: Create Child View

`resources/views/home.blade.php`

```
@extends('layouts.app')
```

```
@section('title')
```

```
  Home Page
```

```
@endsection
```

```
@section('content')
```

```
  <h2>Welcome to Home Page</h2>
```

```
@endsection
```

`@extends` → inherit layout

`@section` → fill content

STEP 3: Route + Controller

```
Route::get('/', function () {
  return view('home');
```

```
});
```

OUTPUT

Final rendered page:

My Website

Welcome to Home Page

© 2026 My Website

IMPORTANT BLADE DIRECTIVES

1. @extends

```
@extends('layouts.app')
```

Inherit layout

2. @section

```
@section('content')
```

Data here

```
@endsection
```

Define section content

3. @yield

```
@yield('content')
```

Display section

4. @include

```
@include('partials.header')
```

Include reusable parts (header/footer)

Secure Routing

In **Laravel**, *secure routes* means protecting routes so only authorized or authenticated users can access them. This is mainly done using **middleware**, especially authentication and authorization middleware.

Secure routes are routes that:

- Require **login (authentication)**
- Restrict access based on **roles/permissions**
- Prevent unauthorized users from accessing sensitive pages

Using Middleware for Secure Routes

Laravel provides built-in middleware like:

- **auth** → only logged-in users
- **verified** → email verified users
- **throttle** → rate limiting
- custom middleware → role-based security

Example 1: Secure Route with Authentication

```
use Illuminate\Support\Facades\Route;
```

```
Route::get('/dashboard', function () {  
    return "Welcome to dashboard";  
})->middleware('auth');
```

Only logged-in users can access **/dashboard**

Example 2: Group Secure Routes

```
Route::middleware(['auth']) ->group(function () {
```

```
    Route::get('/profile', function () {  
        return "User Profile";  
    });
```

```
    Route::get('/settings', function () {  
        return "Settings Page";  
    });
```

```
});
```

All routes inside the group are protected

STEP 1: Create Laravel Project (if not already)

```
composer create-project laravel/laravel myApp  
cd myApp
```

STEP 2: Install Authentication (Breeze)

```
composer require laravel/breeze --dev  
php artisan breeze:install  
php artisan migrate  
npm install && npm run dev
```

- This automatically sets up:
 - a. Login / Register pages
 - b. Auth controllers
 - c. **auth** middleware

STEP 3: Run Server

php artisan serve

Open: <http://127.0.0.1:8000>

STEP 4: Add Secure Route

routes/web.php

```
use Illuminate\Support\Facades\Route;
Route::get('/', function () {
    return "Home Page";
});
// ADD THIS ROUTE
Route::get('/dashboard', function () {
    return "Welcome Dashboard";
})->middleware(['auth'])->name('dashboard')
```

STEP 5: Test Auth Middleware

Case 1: Not Logged In

Open: <http://127.0.0.1:8000/dashboard>

Output: Redirect to **/login**

Case 2: Logged In

Open: <http://127.0.0.1:8000/register>

- Create account
- Login
- Open **/dashboard**

Output:

This is Secure Dashboard

STEP 6: Using Controller

Create Controller

```
php artisan make:controller DashboardController
```

app/Http/Controllers/DashboardController.php

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class DashboardController extends Controller{
    public function __construct(){
        // Apply auth middleware
        $this->middleware('auth');
    }
    public function index(){
        return "Secure Dashboard from Controller";
    }
}
```

Add Route

routes/web.php

```
use App\Http\Controllers\DashboardController;  
Route::get('/dashboard', [DashboardController::class, 'index']);
```

STEP 7: Route Group (Best Practice)

```
Route::middleware(['auth']) -> group(function () {  
    Route::get('/profile', function () {  
        return "Profile Page";  
    });  
    Route::get('/settings', function () {  
        return "Settings Page";  
    });  
});
```

Domain Routing?

Domain routing allows you to handle **different domains or subdomains** in the same Laravel app.

example.com → Main site
admin.example.com → Admin panel
api.example.com → API routes

Basic Domain Routing

`routes/web.php`

```
Route::domain('admin.localhost') ->group(function () {  
    Route::get('/', function () {  
        return "Admin Panel";  
    });  
});
```

Access in Browser

<http://admin.localhost:8000>

You will see: **Admin Panel**

Important Setup : Add domain in hosts file

1. Windows: C:\Windows\System32\drivers\etc\hosts
2. Mac/Linux: /etc/hosts

Add: 127.0.0.1 admin.localhost

Multiple Domain Routing

```
Route::domain('admin.localhost') ->group(function () {  
    Route::get('/dashboard', function () {  
        return "Admin Dashboard";  
    });  
});
```

```
Route::domain('user.localhost') ->group(function () {  
    Route::get('/dashboard', function () {  
        return "User Dashboard";  
    });  
});
```


Dynamic Subdomain Routing

```
Route::domain('{account}.localhost') ->group(function () {  
    Route::get('/', function ($account) {  
        return "Welcome " . $account;  
    });  
});
```

Output

http://abc.localhost:8000 → Welcome abc

http://xyz.localhost:8000 → Welcome xyz

Using Controller (Optional)

```
Route::domain('admin.localhost') ->group(function () {  
    Route::get('/home', [HomeController::class, 'home']);  
});
```

Route Groups

Route groups allow you to **apply common properties** (middleware, prefix, namespace, etc.) to multiple routes.

Basic Syntax

```
Route::group([], function () {  
  
    Route::get('/home', function () {  
        return "Home";  
    });  
  
    Route::get('/about', function () {  
        return "About";  
    });  
});
```

Example with Middleware

```
Route::middleware(['age.check']) ->group(function () {  
    Route::get('/home', function () {  
        return "Home Page";  
    });  
    Route::get('/dashboard', function () {  
        return "Dashboard";  
    });  
});
```

Middleware applies to **all routes inside the group**.

Request → Group Middleware → Route → Response

Route Prefixing

Adds a **common URL path** before all routes in a group.

Basic Prefix Example

```
Route::prefix('admin')->group(function () {  
    Route::get('/dashboard', function () {  
        return "Admin Dashboard";  
    });  
    Route::get('/users', function () {  
        return "Admin Users";  
    });  
});
```

Output URLs

/admin/dashboard

/admin/users

Combine Group + Prefix + Middleware

```
Route::prefix('admin')  
    ->middleware(['age.check'])  
    ->group(function () {  
        Route::get('/dashboard', function () {  
            return "Admin Dashboard";  
        });  
        Route::get('/settings', function () {  
            return "Admin Settings";  
        });  
    });
```

/admin → admin panel

/user → user panel

/api → API routes

Nested Groups (Advanced)

```
Route::prefix('admin')->group(function () {  
    Route::prefix('users') ->group(function () {  
        Route::get('/', function () {  
            return "All Users";  
        });  
        Route::get('/create', function () {  
            return "Create User";  
        });  
    });  
});  
  
/admin/users  
/admin/users/create
```

```
Route::prefix('admin')->middleware(['age.check']) ->group(function () {  
    Route::get('/dashboard', function () {  
        return "Dashboard";  
    });  
});
```

URL Generation:

URL generation means creating **links (URLs)** to routes, controllers, or assets using Laravel helper functions instead of writing hardcoded URLs.

- URL generation in Laravel is the process of dynamically creating URLs using helper functions like `url()`, `route()`, and `asset()`.
- Avoid hardcoding URLs
- Easy maintenance
- It works even if route changes
- Cleaner code

1. Using `url()` Helper

```
$url = url('/home');
```

Output:

```
http://127.0.0.1:8000/home
```


Using **route()** Helper

Step 1: Define Named Route

```
Route::get('/home', function () {  
    return "Home";  
})->name('home');
```

Step 2: Generate URL

```
$url = route('home');
```

Output:

<http://127.0.0.1:8000/home>

Route with Parameters

```
Route::get('/user/{id}', function ($id) {  
    return $id;  
})->name('user');
```

Generate URL

```
$url = route('user', ['id' => 5]);
```

Output: /user/5

Using **asset()** (for CSS, JS, images)

```
echo asset('css/style.css');
```

Output: http://127.0.0.1:8000/css/style.css

Using **action()** (Controller URL)

```
use App\Http\Controllers\HomeController;  
$url = action([HomeController::class, 'home']);
```

In Blade

```
<a href="{{ url('/home') }}">Home</a>
```

```
<a href="{{ route('home') }}">Home</a>
```

```
<a href="{{ route('user', 5) }}">User</a>
```

```
<link rel="stylesheet" href="{{ asset('css/style.css') }}"
```

1. Current URL:

- To retrieve and display the current URL, you can do the following:
- Open an existing Blade view file or create a new one (e.g., resources/views/current-url.blade.php).
- In your Blade view file, add the following code to display the current URL.
- In current.blade.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Current URL Example</title>
</head>
<body>
    <p>Current URL: {{ url()->current() }}</p>
    <p>Current URL: {{ url()->previous() }}</p>
    <p>Current URL: {{ url()->full() }}</p>
</body>
</html>
```

In web.php and in UserController

In web.php

```
// Route to display the current URL example
```

```
Route::get('/current-url', [UserController::class, 'currentUrl']);
```

UserController function

```
public function currentUrl()  
{  
    return view('current');  
}
```

Browser Output

Current URL: <http://localhost:8000/current-url>

2. Generating Framework URLs:

- To generate framework URLs for named routes and controller actions, follow these steps:
- Open an existing Blade view file or create a new one (e.g., `resources/views/framework-urls.blade.php`).
- In your Blade view file, you can generate URLs using the `route()` and `action()` functions. For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Framework URLs Example</title>
</head>
<body>
    @php
        use App\Http\Controllers\UserController;
    @endphp

    <p>Profile URL: <a href="{{ route('profile') }}">Profile</a></p>
    <p>User Edit URL: <a href="{{ action([UserController::class,
'showProfile']) }}">My Profile</a></p>
</body>
</html>
```

3. Asset URLs:

To generate URLs for asset files (CSS, JavaScript, images), follow these steps:

```
<!DOCTYPE html>
<html>
<head>
  <title>Asset URLs Example</title>
  <link rel="stylesheet" href="{{ asset('css/style.css') }}">
</head>
<body>
  <!-- Your HTML content here -->
</body>
</html>
```


4. Generation Shortcuts:

Laravel provides shortcuts for generating URLs directly in Blade templates. You can use these shortcuts within your Blade views. For example:

```
<!DOCTYPE html>
<html>
<head>
    <title>Shortcuts Example</title>
</head>
<body>
    <p>About Us: <a href="{{ url('/admin/dashboard') }}">AboutUs</a></p>
    <p>Profile: <a href="{{ route('profile') }}">Profile</a></p>
</body>
</html>
```